

Jobsheet - Clustering

1.1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

- Mengetahui tentang konsep Clustering
- Mengimplementasikan Clustering menggunakan Python

1.2. Peralatan yang dibutuhkan

Beberapa peralatan yang dibutuhkan dalam menyelesaikan praktikum ini adalah:

- Dataset terkait

1.3. Dasar Teori

1.3.1. Clustering

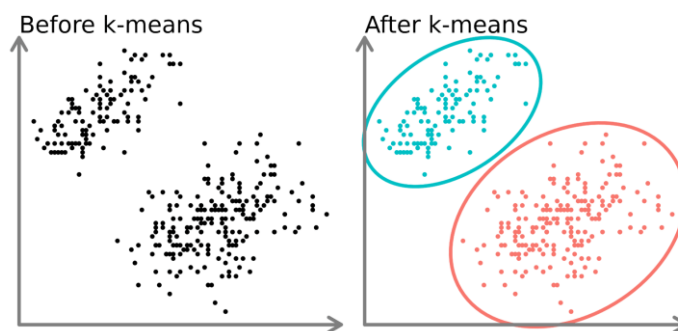
Clustering adalah teknik analisis data yang digunakan untuk mengelompokkan objek atau pengamatan ke dalam kelompok atau klaster yang berbeda, di mana objek dalam klaster yang sama memiliki kesamaan tertentu dan objek di klaster yang berbeda memiliki perbedaan yang signifikan. Tujuan utama dari clustering adalah untuk mengidentifikasi struktur yang tersembunyi atau pola dalam dataset yang tidak terlabel, serta memahami karakteristik alami dari data.

Proses clustering dimulai dengan dataset yang terdiri dari sejumlah objek atau pengamatan, di mana setiap objek direpresentasikan oleh serangkaian atribut atau fitur. Teknik clustering kemudian mencoba untuk menemukan struktur dalam data berdasarkan kesamaan atau jarak antara objek-objek tersebut.

Terdapat berbagai pendekatan yang dapat digunakan dalam clustering, diantaranya:

1. **K-means:** Pendekatan ini mengelompokkan objek berdasarkan pusat kluster yang berbeda. Objek-objek ditempatkan dalam kluster yang memiliki pusat terdekat dengan objek tersebut.

K-Means Clustering adalah salah satu teknik clustering yang paling umum dan sederhana yang digunakan dalam analisis data. Teknik ini bertujuan untuk mengelompokkan titik-titik data ke dalam k kelompok atau kluster berdasarkan kesamaan antara titik-titik data tersebut. K dalam K-Means mewakili jumlah kluster yang diinginkan dan harus ditentukan sebelumnya oleh pengguna.

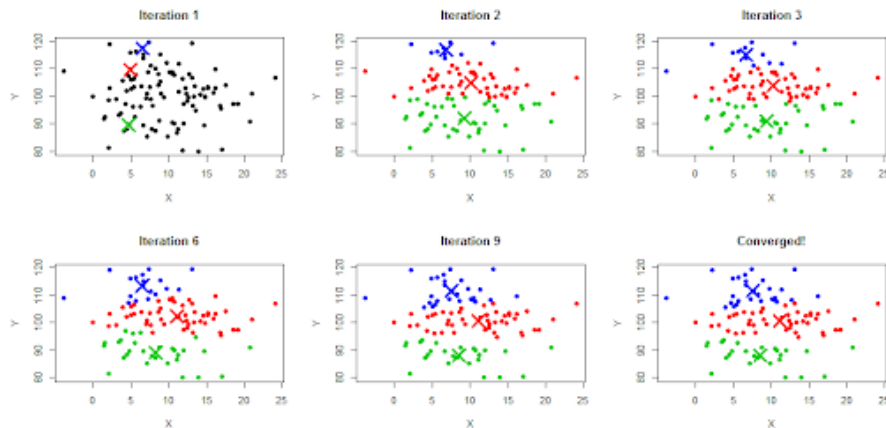


Gambar 1. Ilustrasi Klasterisasi Data Menggunakan K-Means

Berikut adalah langkah-langkah utama dalam K-Means Clustering:

1. **Inisialisasi Pusat Klaster (Centroid):** Langkah pertama dalam K-Means adalah menginisialisasi pusat kluster secara acak. Pusat kluster awal ini dapat dipilih dari titik-titik data secara acak atau dengan menggunakan pendekatan lain seperti K-Means++ yang berusaha untuk memilih pusat kluster awal secara cerdas untuk mempercepat konvergensi.
2. **Pengelompokan Data:** Setelah inisialisasi pusat kluster, setiap titik data ditempatkan dalam kluster yang memiliki pusat terdekat dengan titik data tersebut. Jarak antara titik data dan pusat kluster dapat dihitung menggunakan metrik jarak Euclidean atau metrik jarak lainnya.
3. **Perbarui Pusat Klaster:** Setelah pengelompokan awal, pusat kluster diperbarui dengan menghitung rata-rata dari semua titik data dalam setiap kluster. Pusat kluster baru ini kemudian digunakan sebagai pusat untuk kluster pada iterasi berikutnya.
4. **Iterasi:** Langkah pengelompokan dan pembaruan pusat kluster dilakukan secara berulang hingga kriteria berhenti tercapai. Kriteria berhenti ini bisa berupa jumlah iterasi maksimum yang telah dicapai, perubahan pusat kluster yang tidak signifikan antara iterasi, atau kriteria lainnya.

5. **Evaluasi Hasil:** Setelah konvergensi, hasil clustering dievaluasi menggunakan metrik evaluasi seperti SSE (Sum of Squared Errors) atau Silhouette Score untuk mengevaluasi kualitas kluster yang dihasilkan.



Gambar 2. Tahapan Clustering menggunakan K-Means

Beberapa hal yang perlu diperhatikan dalam K-Means Clustering adalah:

- K-Means sangat sensitif terhadap inisialisasi pusat kluster awal. Hasil clustering dapat bervariasi tergantung pada inisialisasi ini.
- K-Means biasanya lebih baik digunakan untuk data yang berbentuk bulat dan memiliki varian yang seragam dalam setiap dimensinya.
- K-Means tidak dapat menangani kluster dengan bentuk yang kompleks atau tidak teratur.
- K-Means dapat memiliki masalah dengan kluster berukuran yang tidak seimbang atau dengan keberadaan outlier.

Meskipun sederhana, K-Means Clustering tetap menjadi salah satu algoritma clustering yang paling sering digunakan karena efisiensinya dalam menangani data yang berukuran besar dan kemampuannya untuk menghasilkan kluster yang jelas dan terdefinisi dengan baik.

2. **Hierarchical clustering:** Teknik ini menghasilkan dendrogram yang menunjukkan hubungan hierarkis antara kluster. Objek-objek dikelompokkan berdasarkan kedekatannya satu sama lain, dan kluster yang serupa digabungkan secara bertahap. Berikut adalah langkah-langkah utama dalam hierarchical clustering menggunakan pendekatan agglomerative:

1. **Inisialisasi Klaster:** Setiap titik data awalnya dianggap sebagai klaster tunggal.
2. **Perhitungan Jarak Antar-Klaster:** Jarak antara setiap pasangan klaster dihitung. Berbagai metrik jarak dapat digunakan, seperti jarak Euclidean, jarak Manhattan, atau jarak Mahalanobis.
3. **Penggabungan Klaster:** Dua klaster yang memiliki jarak terdekat digabung menjadi satu klaster baru. Proses ini diulangi hingga semua titik data tergabung dalam satu klaster besar.
4. **Pembentukan Dendrogram:** Dendrogram dibentuk untuk merepresentasikan struktur hierarkis dari klaster. Dendrogram adalah representasi grafis dari proses penggabungan klaster, di mana sumbu vertikal menunjukkan tingkat kesamaan antar-klaster dan sumbu horizontal menunjukkan klaster atau titik data mana yang digabung pada setiap langkah.
5. **Memilih Jumlah Klaster:** Setelah dendrogram terbentuk, pengguna dapat memilih jumlah klaster dengan memotong dendrogram pada tingkat yang sesuai dengan tujuan analisisnya.

Hierarchical clustering memiliki beberapa kelebihan dan kekurangan:

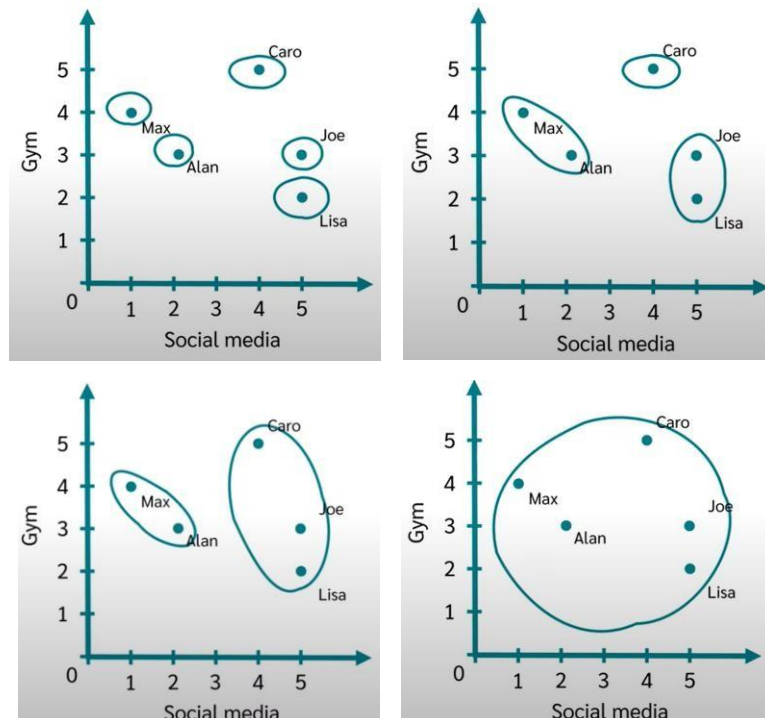
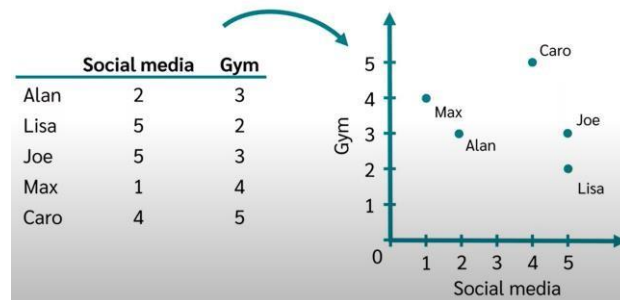
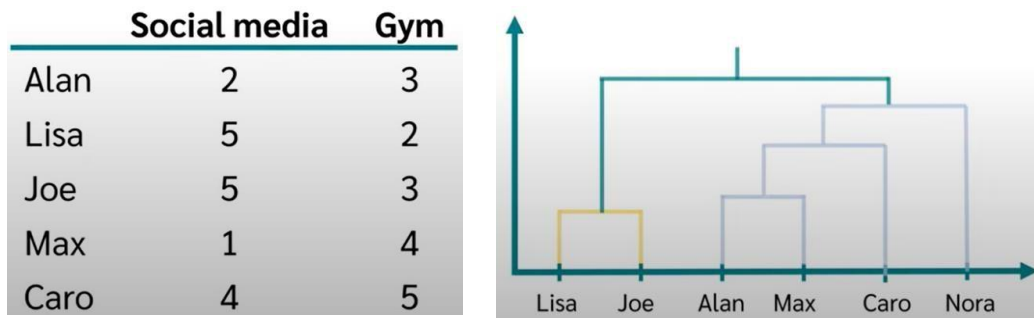
- **Kelebihan:**

- Tidak memerlukan jumlah klaster yang ditentukan sebelumnya, sehingga cocok untuk dataset di mana jumlah klaster tidak diketahui.
- Memungkinkan visualisasi struktur hierarkis dari klaster menggunakan dendrogram.
- Mampu menangani klaster dengan bentuk yang kompleks atau tidak teratur.

- **Kekurangan:**

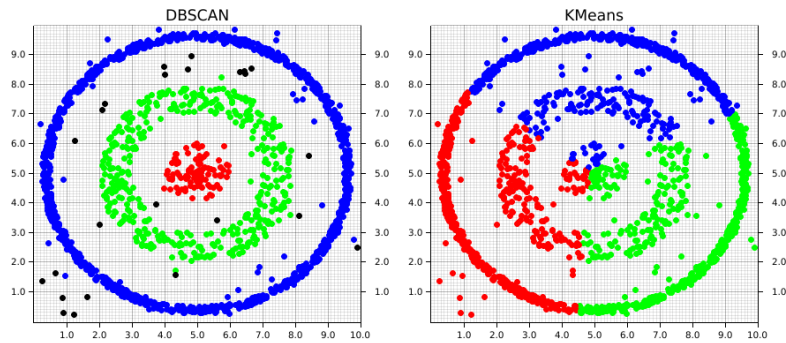
- Membutuhkan waktu komputasi yang lebih lama, terutama untuk dataset yang besar.
- Sensitif terhadap metrik jarak dan metode penggabungan klaster.
- Memerlukan pemilihan parameter yang tepat, seperti metode penggabungan klaster dan kriteria pemotongan dendrogram.

Meskipun memiliki keterbatasan tertentu, hierarchical clustering tetap menjadi salah satu algoritma clustering yang berguna dan populer karena kemampuannya untuk menghasilkan hierarki klaster yang dapat memberikan wawasan yang berharga tentang struktur data.



Gambar 3. Tahapan pada Hierarchical Clustering

3. **Density-based clustering:** Density-Based Clustering merupakan teknik clustering yang mengelompokkan titik-titik data berdasarkan kepadatan relatif mereka dalam ruang fitur. Teknik ini berfokus pada identifikasi wilayah yang padat dalam data, di mana titik-titik data yang berdekatan dan memiliki kepadatan yang tinggi dianggap sebagai bagian dari kluster, sementara titik-titik data yang terisolasi dianggap sebagai noise atau outlier.



Gambar 4. Tampilan Perbedaan Hasil Menggunakan K-Means dan DBScan

Salah satu algoritma density-based clustering yang paling umum digunakan adalah DBSCAN (Density-Based Spatial Clustering of Applications with Noise). Berikut adalah prinsip kerja DBSCAN:

1. **Pengelompokan Berdasarkan Kepadatan:** DBSCAN mengidentifikasi wilayah yang padat dalam data dengan menentukan titik-titik inti (core points). Sebuah titik inti dianggap sebagai bagian dari kluster jika terdapat jumlah tetangga minimum (MinPts) di sekitarnya dalam jarak tertentu (epsilon).
2. **Eksansi Klaster:** Setiap titik inti dan tetangganya yang berdekatan di dalam wilayah kepadatan yang sama dianggap sebagai bagian dari kluster yang sama. Titik-titik yang berdekatan tetapi bukan titik inti atau tetangga dari titik inti lain dianggap sebagai batas (border points) dan juga dimasukkan ke dalam kluster yang sesuai.
3. **Penghapusan Noise:** Titik-titik yang tidak termasuk dalam wilayah kepadatan dan tidak memiliki tetangga yang cukup banyak dianggap sebagai noise dan dikeluarkan dari proses clustering.

Keuntungan utama dari density-based clustering, khususnya DBSCAN, adalah kemampuannya untuk menangani kluster dengan bentuk yang kompleks dan tidak teratur, serta mampu mengatasi noise dalam data. Selain itu, DBSCAN tidak memerlukan jumlah kluster yang ditentukan sebelumnya dan dapat bekerja efisien dengan dataset besar. DBSCAN juga memiliki beberapa kelemahan, termasuk sensitif terhadap parameter seperti epsilon dan MinPts, serta kinerjanya yang kurang baik dalam menangani dataset dengan perbedaan kepadatan yang signifikan di dalamnya.

Meskipun demikian, density-based clustering tetap menjadi pilihan yang kuat untuk aplikasi di mana struktur kluster yang kompleks dan adanya noise dalam data merupakan tantangan utama, seperti dalam analisis data spasial, deteksi anomali, dan segmentasi objek dalam citra.

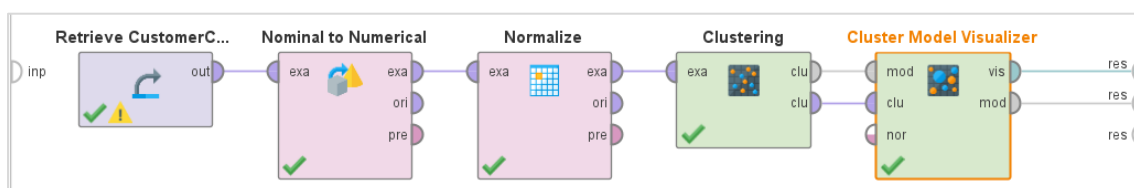
Clustering memiliki berbagai aplikasi dalam berbagai bidang, termasuk pengelompokan konsumen dalam analisis pasar, pengelompokan dokumen dalam penggalian data teks, pengelompokan pasien dalam analisis kesehatan, dan pengelompokan galaksi dalam astronomi. Ini adalah alat yang berguna untuk menganalisis dan memahami struktur data yang kompleks dan besar, serta mengidentifikasi pola-pola yang bermanfaat dalam dataset yang tidak terlabel.

Percobaan K-Means menggunakan Rapidminer

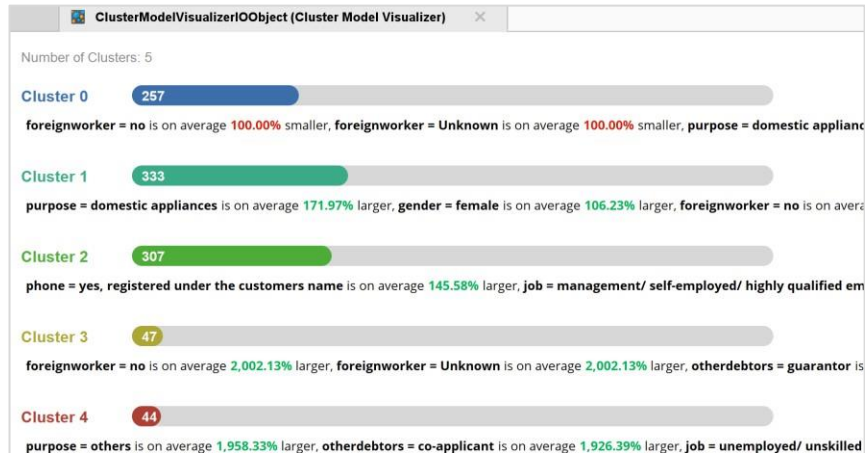
1. Pada percobaan ini akan dilakukan klasterisasi data Resiko Kredit calon nasabah. Data dapat diambil dari Repository → Training Resources → Data → Credit Risk → CustomerCreditRiskData. Data ini memiliki 988 sampel, dengan 21 atribut reguler dan 1 atribut label (creditworthy). Label tidak digunakan pada proses perhitungan clustering.

Row No.	creditworthy	foreignworker	status	credithistory	purpose	savings	employe...	otherdebtors
1	Worthy	yes	< 0 Euro	critical account/	radio/television	unknown/ no...	.. >= 7 years	none
2	Not Worthy	yes	< 200 Euro	existing cred...	radio/television	... < 100 Euro	1 <= ... < 4 y...	none
3	Worthy	yes	no checking ...	critical account/	education	... < 100 Euro	4 <= ... < 7 y...	none
4	Worthy	yes	< 0 Euro	existing cred...	furniture/equ...	... < 100 Euro	4 <= ... < 7 y...	guarantor
5	Not Worthy	yes	< 0 Euro	delay in payi...	car (new)	... < 100 Euro	1 <= ... < 4 y...	none
6	Worthy	yes	no checking ...	existing cred...	education	unknown/ no...	1 <= ... < 4 y...	none
7	Worthy	yes	no checking ...	existing cred...	furniture/equ...	500 <= ... < ...	.. >= 7 years	none
8	Worthy	yes	< 200 Euro	existing cred...	car (used)	... < 100 Euro	1 <= ... < 4 y...	none

2. Perhatikan bahwa beberapa fitur pada dataset masih termasuk nominal atribut. Oleh karena itu, langkah berikutnya adalah menambahkan block operator **Nominal to Numerical**. Operator ini digunakan untuk mengubah seluruh nilai atribut ke dalam numerik agar dapat dilakukan proses perhitungan jarak antardata.
3. Selanjutnya ditambahkan operator **Normalize** menggunakan Z-Transform untuk melakukan normalisasi data.
4. Operator **Clustering** ditambahkan dengan menggunakan jumlah k default yaitu k=5
5. Langkah terakhir tambahkan operator **Cluster Model Visualizer** untuk menampilkan hasil Clustering.



6. Perhatikan hasil setelah dirunning, terdiri dari 5 cluster dengan cluster 0 berisi 257 sampel data dan seterusnya.

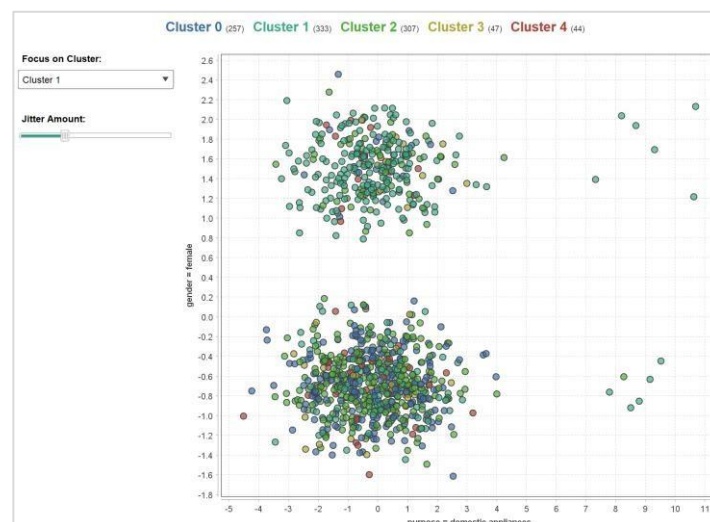


7. Centroid untuk masing-masing Cluster dapat dilihat pada Centroid Table

Cluster	foreign...	foreign...	foreign...	status ...	status ...	status ...	status ...	credith...	credith...	credith...	credith...	
Cluster 0	0.223	-0.194	-0.106	-0.059	-0.078	0.148	-0.046	0.275	-0.417	0.147	0.091	C
Cluster 1	0.223	-0.194	-0.106	0.116	0.120	-0.261	0.093	-0.390	0.524	-0.202	-0.038	~
Cluster 2	0.223	-0.194	-0.106	-0.150	-0.017	0.185	-0.067	0.161	-0.210	0.130	-0.007	~
Cluster 3	-4.472	3.891	2.123	0.241	-0.365	0.063	0.097	0.201	-0.042	-0.158	0.010	~
Cluster 4	0.223	-0.194	-0.106	0.249	0.061	-0.249	-0.068	0.009	-0.017	-0.067	-0.205	C

8. Hasil Cluster juga dapat divisualisasikan menggunakan Scatter Plot seperti gambar berikut. Jika titik data hanya sedikit, kemungkinan banyak titik data yang menumpuk di posisi yang sama. Geser jitter amount hingga data terlihat. Jitter amount adalah besar kecilnya gangguan acak (noise) yang ditambahkan ke posisi titik pada visualisasi (biasanya scatter plot) agar titik yang saling bertumpuk bisa terlihat.

Hasil cluster terlihat tumpang tindih karena visualisasi hanya menggunakan 2 fitur sementara k-means dilakukan terhadap data dengan banyak fitur.



Percobaan K-Means menggunakan Python

1. Pada percobaan ini akan dilakukan klasterisasi data Resiko Kredit calon nasabah, file `credit_data_sample.csv` sudah disertakan.

Age	Credit_amount	Duration	Employment_since	Purpose
25	2000	12	2	car
40	7000	48	5	business
35	3500	24	4	car
23	1200	10	1	furniture
52	9000	60	5	real_estate
46	8000	54	4	real_estate
28	2500	18	2	car

Lakukan import Library yang dibutuhkan:

```
# Import library yang diperlukan
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

2. Langkah berikutnya adalah meload data dari Google Drive (*silakan menyesuaikan path atau gunakan metode import dataset seperti jobsheet sebelumnya*)

```
# Load data
df =
pd.read_csv('/content/drive/MyDrive/Kuliah/2025 Data
Mining/Minggu 10/credit_data_sample.csv', sep=';')
# Tampilkan 5 data pertama
print(df.head())
```

	Age	Credit_amount	Duration	Employment_since	Purpose
0	25	2000	12	2	car
1	40	7000	48	5	business
2	35	3500	24	4	car
3	23	1200	10	1	furniture
4	52	9000	60	5	real_estate

3. Selanjutnya kita akan pilih fitur untuk Clustering, untuk saat ini kita pilih fitur numerik saja.

```
features = ['Age', 'Credit_amount', 'Duration',
'Employment_since']
X = df[features]
```

4. Karena skala antara fitur berbeda-beda, maka kita lakukan standardisasi data.

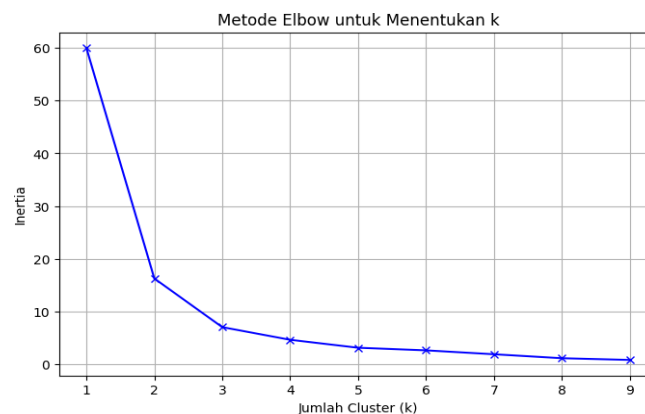
```
# Standarisasi data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

5. Langkah berikutnya adalah menentukan jumlah cluster. Jumlah cluster dapat ditentukan sesuai kebutuhan kita, atau dapat juga dicari jumlah cluster optimal menggunakan Elbow Method.

```
# Elbow Method
inertia = []
K = range(1, 10)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot hasil elbow
plt.figure(figsize=(8,5))
plt.plot(K, inertia, 'bx-')
plt.xlabel('Jumlah Cluster (k)')
plt.ylabel('Inertia')
plt.title('Metode Elbow untuk Menentukan k')
plt.grid(True)
plt.show()
```



6. Langkah berikutnya kita lakukan K-Means Clustering dengan 3 Cluster.

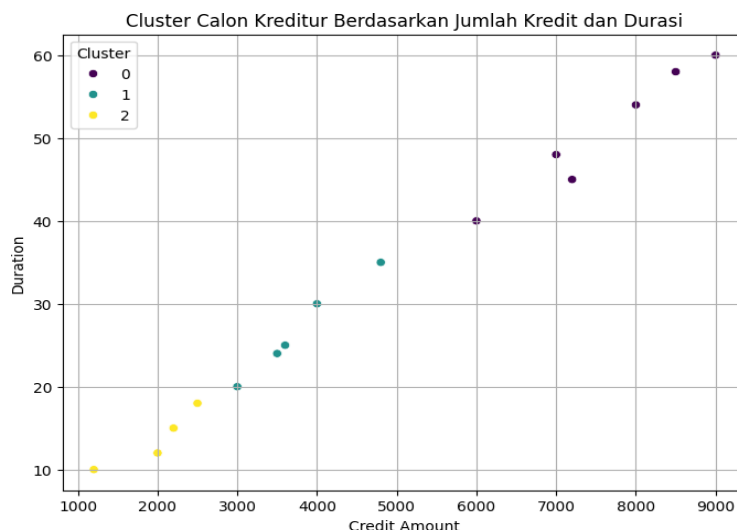
```
# Buat model KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# Lihat hasil clustering
print(df.head())
```

	Age	Credit_amount	Duration	Employment_since	Purpose	Cluster
0	25	2000	12	2	car	2
1	40	7000	48	5	business	0
2	35	3500	24	4	car	1
3	23	1200	10	1	furniture	2
4	52	9000	60	5	real_estate	0

7. Visualisasi hasil Cluster dapat dijalankan dengan Code berikut.

```
# Visualisasi hasil clustering
plt.figure(figsize=(8,6))
sns.scatterplot(x=df['Credit_amount'], y=df['Duration'],
hue=df['Cluster'], palette='viridis')
plt.title('Cluster Calon Kreditur Berdasarkan Jumlah Kredit
dan Durasi')
plt.xlabel('Credit Amount')
plt.ylabel('Duration')
plt.grid(True)
plt.show()
```



Kita dapat lakukan interpretasi hasil Clustering tiap cluster sebagai berikut:

- Cluster 0** = Risiko rendah → kredit kecil, durasi pendek.
- Cluster 1** = Risiko sedang → kredit sedang, durasi menengah.
- Cluster 2** = Risiko tinggi → kredit besar, durasi panjang.

8. Berikutnya kita akan lihat performa dari K-Means model menggunakan Silhouette Score

```
# 7a. Hitung Silhouette Score
silhouette_avg = silhouette_score(X_scaled,
df['Cluster'])
print(f"\nSilhouette Score: {silhouette_avg:.3f}")

# 7b. Tampilkan inertia akhir
print(f"Inertia (k=3): {kmeans.inertia_:.3f}")

# Step 8 – Visualisasi hasil clustering (2 fitur:
Credit_amount vs Duration)
plt.figure(figsize=(8,6))
sns.scatterplot(
    x=df['Credit_amount'],
    y=df['Duration'],
    hue=df['Cluster'],
    palette='viridis',
    s=100
)
plt.title('Cluster Calon Kreditur Berdasarkan Credit
Amount dan Duration')
plt.xlabel('Credit Amount')
plt.ylabel('Duration')
plt.legend(title='Cluster')
plt.grid(True)
plt.show()
```

Silhouette Score: 0.549
Inertia (k=3): 7.016

9. Semakin kecil nilai Inertia, maka semakin kecil jarak antar data dalam satu cluster, nilai ini terlihat dari grafik Elbow Method. Silhouette Score (SC) menghitung seberapa mirip objek dengan clusternya sendiri dibandingkan dengan cluster lain. Nilainya antara -1 sampai 1. Score 0,549 sudah dianggap hasil yang cukup baik. Cluster sudah cukup terpisah tetapi belum sempurna.

Nilai Score	Interpretasi
~ 1.0	Cluster sangat baik (data sangat cocok dengan clusternya sendiri, dan jauh dari cluster lain)
~ 0.5	Cluster cukup baik (struktur cluster lumayan jelas)
~ 0.0	Cluster saling tumpang tindih (overlapping), kurang jelas
~ < 0.0	Salah cluster, data lebih cocok di cluster lain

Tugas:

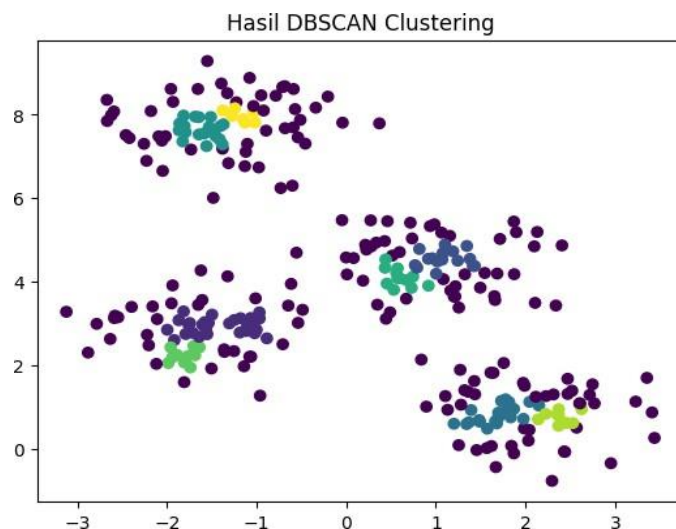
1. Lakukan K-Means Clustering pada Data Iris (Repository → Samples → Data → Iris). Buat grafik elbow method untuk mengetahui nilai k yang optimal. Tentukan nilai silhouette score nya. Screenshot grafiknya dan tuliskan nilai silhouette score.
2. Lakukan percobaan clustering menggunakan DBSCAN pada contoh data.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs

# Membuat data contoh
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60,
random_state=0)

# Menggunakan DBSCAN
dbscan = DBSCAN(eps=0.3, min_samples=10)
labels = dbscan.fit_predict(X)

# Visualisasi hasil clustering
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("Hasil DBSCAN Clustering")
plt.show()
```

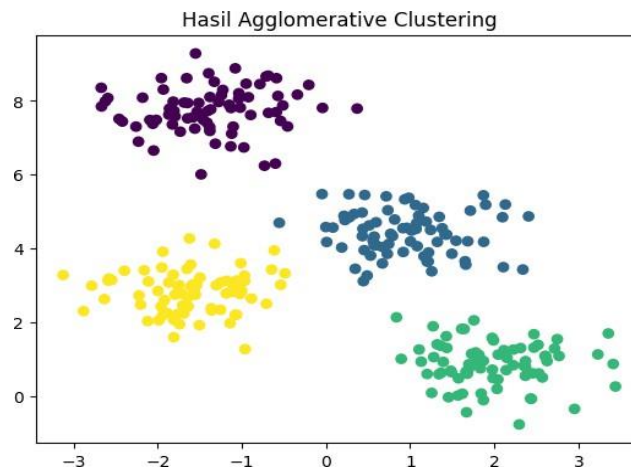


Lakukan percobaan clustering menggunakan Agglomerative Clustering pada contoh data yang sama dengan No 2.

```
from sklearn.cluster import AgglomerativeClustering

# Menggunakan Agglomerative Clustering
agg_clust = AgglomerativeClustering(n_clusters=4)
agg_labels = agg_clust.fit_predict(X)

# Visualisasi hasil clustering
plt.scatter(X[:, 0], X[:, 1], c=agg_labels,
            cmap='viridis')
plt.title("Hasil Agglomerative Clustering")
plt.show()
```



Perhatikan bahwa hasil clustering untuk data contoh dengan DBScan tidak sebaik dengan Agglomerative Clustering. Lakukan analisa, mengapa demikian? Coba lakukan perubahan parameter eps (epsilon) dan min_samples (minPts) dengan beberapa kombinasi nilai agar cluster yang dihasilkan lebih baik. Screenshot hasil scatter plot clustering dengan DBScan yang sudah diperbaiki dan tuliskan nilai eps dan min_samples terbaik yang dapat Anda temukan.

--- SELAMAT BELAJAR ---